

Prototypical Networks for Few-shot Learning

Contents

1	Introduction	2
2	Related Work	3
3	Theory	4
4	Few-Shot Prediction	6
5	Zero-Shot Prediction	8
6	Mahalanobis Distance	11
7	Prototype Refinement	14
8	Discussion	18

1 Introduction

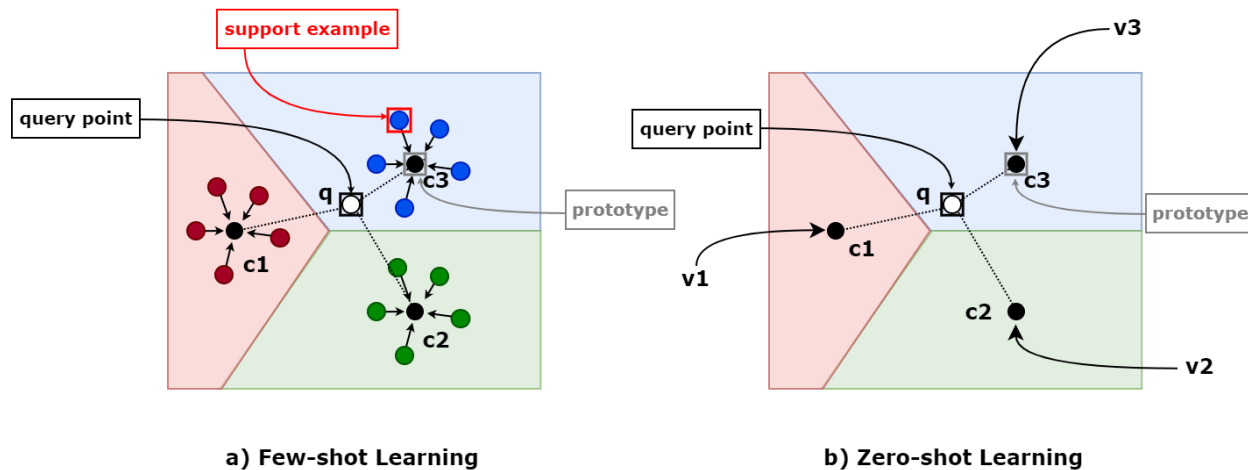


Figure 1: Few-shot (left) and zero-shot (right) classification using Prototypical Networks: In the few-shot scenario, the prototype c_k of each class k is calculated as the mean of the support vectors in the embedding space (left). In the zero-shot case (right) the prototypes c_k are computed using embeddings of meta-data v_k . A query point q is then assigned a class using a softmax over Euclidean distances to class prototypes. (Adapted from (Snell, Swersky, and Zemel 2017))

Deep Learning has achieved promising results in image classification, approaching human-like performance (Ciregan, Meier, and Schmidhuber 2012). Yet, it relies on large annotated datasets (Simonyan and Zisserman 2014; Krizhevsky, Sutskever, and Hinton 2012) which are costly to obtain and makes the training expensive. Humans, on the other hand, excel at recognizing and learning new concepts rapidly with scarce data (B. Lake et al. 2011). When visiting a foreign country, for example, humans can quickly learn to distinguish different types of vegetables. This motivates the development of Machine Learning methods capable of classifying new instances with little annotated data.

Few-shot classification aims to generalise to new classes after seeing limited examples per class (Vinyals et al. 2016; Snell, Swersky, and Zemel 2017; Finn, Abbeel, and Levine 2017; Ravi and Larochelle 2016). Diverse solutions have been proposed, ranging from model-based and optimisation-based (Ravi and Larochelle 2016), to distance-metrics-based, (Snell, Swersky, and Zemel 2017) methods. Among the latter, Prototypical Networks (Snell, Swersky, and Zemel 2017) with their simple but effective approach of learning a mapping between images and an embedding space and assigning the class of the closest neighbor to a new image, shows promising results in generalizing fast to new data not seen at training time. Moreover, zero-shot learning, leveraging class meta-data, presents an approach for the extreme case of classifying in the absence of any labeled examples of the target class.

We replicate the results of Snell, Swersky, and Zemel (2017)’s few-shot classification on the Omniglot (B. M. Lake, Salakhutdinov, and Tenenbaum 2015), and miniImageNet (Vinyals et al. 2016) dataset, and test zero-shot classification on the Caltech-UCSD Birds-200-2011 (CUB) Dataset (Welinder et al. 2010). As the original paper represents each prototype by merely using the mean of the distribution, we consider two extensions. First, we explore Gaussian prototypical networks with class-dependent covariance matrices by adopting Mahalanobis distance over Euclidean distance, as proposed by (Fort 2017). Second, building

on the Mahalanobis-distance based approach, we use Transductive Conditional Neural Adaptive Processes (CNAPS) which address the problem of few-shot learning using unlabelled instances and soft k-means clustering updates for the class parameters to jointly classify queries as inspired by Bateni, Barber, et al. (2022). We demonstrate enhanced performance in fewer-shot scenarios when adapting Mahalanobis distance and soft k-means clustering for the prototype refinement, underscoring the importance of metric selection and distribution modelling within the embedding space.

2 Related Work

The quest for efficient few-shot learning algorithms has led to the development of various approaches capable of learning from a limited number of examples. These can be categorized into three main groups (Parnami and Lee 2022):

1. **Distance-metric-based models:** These models, such as Prototypical Networks (Snell, Swersky, and Zemel 2017) and Siamese Networks (Koch, Zemel, Salakhutdinov, et al. 2015), rely on embedding spaces where image similarities are calculated. By comparing distances between two image embeddings, these models efficiently assign labels to new instances based on proximity to known examples, enabling rapid inference with minimal computational overhead.
2. **Optimization-based methods:** LSTM-Meta Learners (Ravi and Larochelle 2016) and Model-Agnostic Meta-Learners (MAML) (Finn, Abbeel, and Levine 2017) focus on finding good model initializations. These methods facilitate quick adaptation to new tasks with few examples by adjusting model parameters using gradient descent, but are however susceptible to overfitting.
3. **Memory-augmented models:** Using an architecture incorporating external memory, such as the Neural Turing Machine (Graves, Wayne, and Danihelka 2014) and Memory-Augmented Neural Networks (Santoro et al. 2016) store and recall task-relevant information.

Among these, distance-metric-based models, comparing new instances with experienced examples, offer a solution that facilitates fast inference and easy implementation. An early approach by Koch, Zemel, Salakhutdinov, et al. (2015) employed convolutional Siamese networks to produce image embeddings for support and query images via a network pair with shared parameters. They classified images based on the closest neighbor as defined by the L1 absolute distance. Matching networks on the other hand learn independent feature-extractors for each query and support examples and compute cosine similarities over feature vectors to obtain nearest neighbors (Vinyals et al. 2016). Relation networks extend the aforementioned paradigms by not relying on a similarity metric but learn a similarity metric function parametrised by a neural network (Sung et al. 2018).

Snell, Swersky, and Zemel (2017) introduced Prototypical Networks for few-shot classification, emphasizing the efficacy of the squared Euclidean distance between a query example and a class prototype, defined as the mean of the support examples. They show that Euclidean distance as a Bregman divergence outperforms other distance metrics like cosine similarity, assuming spherical Gaussian distributions where feature dimensions are uncorrelated and of uniform variance. The authors argue that Bregman divergences are the most appropriate distance measure choice in this setting of using a mean as the prototype. They also claim that the non-linearity of embedding functions allows for enough flexibility to effectively perform few-shot classification, making a more complex/non-linear distance function unnecessary. However, they don't evaluate the performance of other Bregman divergences and this approach has been criticized for its oversimplified

assumptions regarding the feature distribution.

Instead, Bateni, Goyal, et al. (2020) and Fort (2017) show that replacing the Euclidean distance with Bregman divergences and class-specific covariance matrices improves the performance of prototypical networks for few-shot classification. They advocate for the Mahalanobis distance over Euclidean, attributing improved outcomes to the incorporation of class-specific covariance matrices, thereby addressing the limitations of assuming feature independence and uniform variance. Furthermore, Fort (2017) proposes Gaussian Prototypical Networks, where the network encoder not only outputs an embedded vector but also estimates of the covariance matrix components, offering a more nuanced understanding of class distributions. This advancement underscores the critical role of carefully chosen metrics and the modeling of within-class sample distributions, challenging prior assumptions and highlighting the importance of metric selection in embedding space for enhancing few-shot learning.

Furthermore, while the backbones of prototypical networks do not have built-in feature adaptation, new approaches for adapting feature extraction at inference time have been proposed. Conditional Neural Adaptive Processes (CNAPS) as one such approach (Requeima et al. 2019) use a pretrained network with FiLM layers (Perez et al. 2018) with adjustable parameters which allows the network to focus on different features for different tasks. Bateni, Goyal, et al. (2020) extends upon this by reducing the number of trainable parameters with Simple CNAPS, replacing the adaptive linear classifier of CNAPs with a prototypical mechanism that utilizes the Mahalanobis distance. **NOT FINISHED** However, a problem that remains is training under very scarce and few support vectors. Transductive few-shot learning is an approach to extract information from the unlabelled examples, jointly predicting all query examples. Therefore, approaches that make use of the unlabelled information in the query set have been proposed to enhance the performance in few-shot scenarios (Yanbin Liu et al. 2018; Bateni, Barber, et al. 2022). Transductive CNAPS extend the framework of Simple CNAPS, adapting a transductive approach. They consist of a soft-k-means update mechanism using the EM algorithm to update soft labels dynamically for the parameter estimates of the mean and covariance matrix leveraging both unlabelled and labelled data.

3 Theory

3.1 Prototypical Networks

When dealing with few-shot classification using prototypical networks, we have a support set $S = (x_1, y_1) \dots (x_N, y_N)$ consisting of N examples where each $x_i \in R^D$ denotes a D -dimensional feature vector of an example and $y_i \in 1, \dots, K$ is the corresponding label out of K possible classes. S_k is a subset containing all examples of class k .

Prototypical Networks aim to represent each class by an M -dimensional prototype vector $c_k \in R^M$ which is derived through an embedding function $f_\phi : R^D \rightarrow R^M$ with learnable parameters ϕ . Further, each class prototype c_k is calculated as the mean of the embedded support vectors:

$$c_k = \frac{1}{|S_k|} \sum_{(x_i, y_i) \in S_k} f_\phi(x_i) \quad (1)$$

The probability $p_\phi(y = k|x)$ of a query example x being classified into class k is given by a softmax over negative distances to the prototypes. Here d is a distance function $\mathbb{R}^M \times \mathbb{R}^M \rightarrow [0, +\infty)$ which measures similarity.

$$p_\phi(y = k|x) = \frac{\exp(-d(f_\phi(x), c_k))}{\sum_{k'} \exp(-d(f_\phi(x), c_{k'}))} \quad (2)$$

In the original paper the squared Euclidean distance between the embedded support vector $f_\phi(x)$ and the class prototype c_k is used.

$$d(f_\phi(x), c_k) = (f_\phi(x) - c_k)^T (f_\phi(x) - c_k) \quad (3)$$

Learning is performed by minimizing the negative log likelihood for a true class k as a loss function L using Stochastic Gradient Descent (SGD):

$$L(\theta_1) = -\log P_{\theta_1}(y = c|\hat{x}) \quad (4)$$

For each training episode, a randomly selected subset of classes is chosen as the training set. For each selected class, the support set is then chosen as a randomly selected subset of examples, serving as representative samples for learning each class prototype. The remaining examples, not included in the support set, form the query set, used for evaluating the model's ability to generalize to new instances.

3.2 Bregman Divergence

When using Euclidean distance as the distance function to prototypes for soft-max classification of queries, there is a special interpretation of the prototypes as means of Gaussian clusters with identity variance. In this case, when computing the posterior probability of the queries belonging to each of these clusters, the prototypes effectively serve as the centers of the Gaussian distributions representing the clusters.

The Euclidean distance is a specific case of a broader class of distance functions known as Bregman divergences. These divergences, characterized by the general form given in Equation 5, provide a unified framework for measuring the dissimilarity between probability distributions or points in a metric space.

$$d_\phi(z, z') = \phi(z) - \phi(z') - (z - z')^T \nabla \phi(z') \quad (5)$$

Any regular exponential family distribution can be expressed in terms of a uniquely determined regular Bregman divergence, as shown in Equation 6.

$$p_\psi(z | \Theta) = \exp\{-d_\phi(z, \mu(\Theta)) - g_\psi(z)\} \quad (6)$$

When evaluating the posterior probability of a data point against different instances of these distributions, the expression can be written as Equation 7. In the special case where the prior probabilities, π_k , of these instances are equal, the posterior probability simplifies to the prototypical classification in Equation 2.

$$p(y = k | z) = \frac{\pi_k \exp(-d_\phi(z, \mu(\Theta_k)))}{\sum_{k'} \pi_{k'} \exp(-d_\phi(z, \mu(\Theta_{k'})))} \quad (7)$$

This provides a theoretical justification for using the mean of support vectors as the prototype as the empirical mean corresponds to the Maximum Likelihood Estimate (MLE) of the Gaussian Distribution. It is worth

noting that the Cosine distance, unlike the Euclidean distance, is not a Bregman divergence and consequently researchers have found that results are not as good as it lacks the associated probabilistic interpretation.

3.3 Decision Boundaries

When the Euclidean Distance is used as the distance function, the prototypical classification takes on a linear decision boundary. We can show this by deriving the decision boundary of two classes, showing that it is linear with respect to the embedding $f_\phi(x)$.

$$\begin{aligned} d(f_\phi(x), c_k) - d(f_\phi(x), c_{k'}) &= 0 \\ (f_\phi(x) - c_k)^T (f_\phi(x) - c_k) - (f_\phi(x) - c_{k'})^T (f_\phi(x) - c_{k'}) &= 0 \\ 2f_\phi(x)^T (c_{k'} - c_k) + c_k^T c_k - c_{k'}^T c_{k'} &= 0 \end{aligned} \quad (8)$$

Indeed, authors find that despite this just being a linear model, the model is effective as sufficient non-linearities are captured in the embedding function $f_\phi(x)$.

4 Few-Shot Prediction

We explore Few-Shot Prediction on images utilizing two distinct datasets: the Omniglot dataset and the miniImageNet dataset. The Omniglot dataset consists of 1623 handwritten characters from 50 alphabets, with each character represented by 20 unique examples drawn by different individuals. The miniImageNet dataset features 60,000 color images, each measuring 84x84 pixels, organized into 100 classes with 600 examples per class. For both these datasets, we train the same embedding architecture comprising 4 convolutional blocks where each block comprises a 64-filter 3×3 convolution, batch normalization layer, a ReLU nonlinearity, and a 2×2 max-pooling layer. The output is flattened across dimensions into a one-dimensional vector. For Omniglot, we resize inputs to 28x28, with the architecture yielding one output per 64 dimensional channel. On the other hand, for miniImageNet comprising inputs sized 84x84, this results in a 5x5 output per 64 dimensional channel and hence a 1600-dimensional embedding. For embedding both support and query points, the same encoder was used.

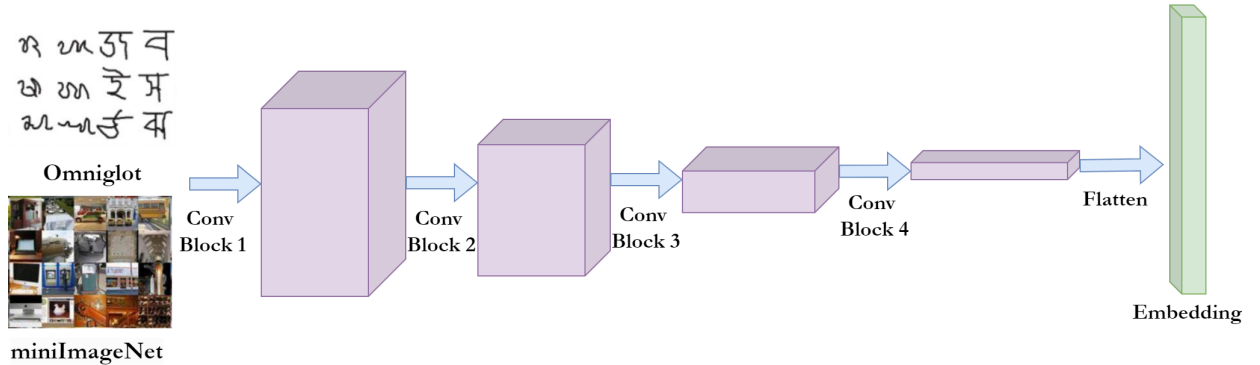


Figure 2: Few Shot Embedding Architecture

For few-shot experiments, we separate the data on a class level into training, validation and test sets. For Omniglot, we begin by augmenting the character classes with rotations in multiples of 90 degrees. We use

4112 classes for training, 688 classes for validation and 1692 classes for testing. For ImageNet, we use the splits by Ravi and Larochelle (2016) with 64 classes for training, 16 classes for validation and 20 classes for testing. We randomly sample episodes and train for a total of 10000 episodes, reporting accuracy on the validation set every 100 episodes. Both models are trained using an initial learning rate of 1e-3 with the learning rate being cut in half every 2000 episodes. To evaluate the model’s performance, we randomly sample 1,000 episodes from the test set for Omniglot and 600 episodes for miniImageNet, and then calculate the average accuracy across these episodes.

To perform these experiments, we leverage an existing codebase provided by Ciriello (2018). It contains the code necessary to prepare and split the Omniglot Dataset, as well as functions to convert datasets into prototypical episode batches, compute prototypes, evaluate the prototypical loss, and define the convolutional backbone. However, there is no associated dataloader for miniImageNet. Indeed, preparing miniImageNet from the full ImageNet dataset is computationally expensive, so we download a cached version of the splits from Yaoyao Liu (2019) and create a custom data loader that can be adapted with the prototypical episode batches function for use in training with the original code.

For Omniglot, we experiment with training on different combinations of several support vectors and classes within each episode in line with the paper. For testing, we follow the paper and also test across 5-way and 20-way episodes with 1-shot and 5-shot. Our results are nearly identical to that of the paper. Trends show that the 5-shot accuracy is consistently higher than the 1-shot accuracy across both 5-way and 20-way classification. This is expected as having more examples or shots per class during training improves the model’s ability to generalize. Again, for both 1-shot and 5-shot scenarios in the table, 5-way accuracies are consistently higher than 20-way accuracies. This trend is also reasonable since it’s generally easier to classify among fewer classes than a large number of classes. Lastly, an increase in the number of ways in the training episodes, resulted in an improvement in both 1-shot and 5-shot accuracies for both 5-way and 20-way classification suggesting that training with a larger number of classes per training episode helps the network to generalize better forcing the model to make more fine-grained decisions in the embedding space.

Distance Metric	Train Episodes			5-way Acc.		20-way Acc.	
	Shot	Query	Way	1-shot	5-shot	1-shot	5-shot
Euclidean	1	15	5	97.4%	99.3%	92.0%	97.6%
Euclidean	1	15	20	98.4%	99.6%	94.8%	98.6%
Euclidean	1	5	60	98.6%	99.6%	95.3%	98.7%
Euclidean	5	15	5	97.5%	99.3%	92.2%	97.9%
Euclidean	5	15	20	98.3%	99.6%	94.4%	98.6%
Euclidean	5	5	60	98.5%	99.6%	94.5%	98.8%

Table 1: Omniglot Results

For miniImageNet, we experiment with the performance of prototypical networks by varying the number of ways per training episode using Euclidian distance measure in Table 2. We follow the same procedure of consistently maintaining the number of query points per class to be 15 during the training episodes as was done in the original paper. The evaluation of classification accuracy was based on the average results from 600 randomly generated episodes from the test set. Results are once again nearly identical to the paper. Trends show that, within each way of setting, 5-shot accuracies are consistently higher than 1-

shot accuracies indicating that in models with more examples per class, the ability to correctly classify new examples increases. In our case, the highest 5-way accuracy for 1-shot learning was observed to be 48.6% with 25-way training episodes and for 5-shot learning, the highest accuracy was 67.7% with 20-way training episodes. These results slightly differ from the original paper where the highest 5-way accuracies using 1-shot and 5-shot classifications were 49.42% and 68.20% using 30-way and 20-way training episodes respectively.

Distance Metric	Train Episodes			5-way Acc.	
	Shot	Query	Way	1-shot	5-shot
Euclidean	1	15	5	44.0%	57.6%
Euclidean	1	15	10	47.1%	62.1%
Euclidean	1	15	15	48.5%	64.0%
Euclidean	1	15	20	48.2%	63.6%
Euclidean	1	15	25	48.6%	64.3%
Euclidean	1	15	30	48.3%	64.3%
Euclidean	5	15	5	46.0%	63.8%
Euclidean	5	15	10	45.6%	65.6%
Euclidean	5	15	15	46.1%	66.9%
Euclidean	5	15	20	45.4%	67.7%
Euclidean	5	15	25	45.3%	67.5%
Euclidean	5	15	30	45.0%	66.9%

Table 2: ImageNet Results

Furthermore, we also experimented with the Cosine Distance measure revealing that the Euclidean distance significantly outperforms the cosine in increasing accuracy as shown in Table 3. This finding reinforces the original paper’s hypothesis that the reduced accuracy with cosine distance is likely due to it not being a Bregman divergence, as the connection to mixture density estimation is not applicable in this case.

Distance Metric	Train Episodes			5-way Acc.	
	Shot	Query	Way	1-shot	5-shot
Cosine	5	15	5	38.80%	45.97%
Euclidean	5	15	5	46.00%	63.80%
Cosine	5	15	20	42.49%	50.80%
Euclidean	5	15	20	45.40%	67.70%

Table 3: ImageNet Cosine Distance

5 Zero-Shot Prediction

We experiment with Zero-Shot prediction on images using the Caltech UCSD Birds Dataset, which contains 11,788 images of 200 bird species. What makes this dataset unique is that it includes not only images but also attributes of the birds they represent, specifically a 312-dimensional vector describing attributes such

as shape, color, and feature patterns. These attributes allow us to perform zero-shot prediction at inference, where we provide the model with only the attributes and no example images, and the model can directly classify the images based on the attributes.

We use the data splits from Reed et al. (2016), where the classes are divided into 100 for training, 50 for validation, and 50 for testing. We use features from the pre-trained GoogleNet to embed the images. During training, we first resize the images to 256x256, then take ten crops of each image: the middle, upper left, upper right, lower left, and lower right crops of both the original and horizontally-flipped image. For test images, we only use the middle crop. We then resize the crops to 224x224, which is the expected input size for GoogleNet. We also normalize the images across the channels with means of [0.485, 0.456, 0.406] and standard deviations of [0.229, 0.224, 0.225], as required for GoogleNet. To bring these embeddings into the same domain as the attributes, we train a network to learn two linear mapping functions that project both the embeddings and attributes into a common space for comparison. We treat the embedded attributes as the prototypes and the embedded images as the queries for classification, as illustrated in Figure 5.

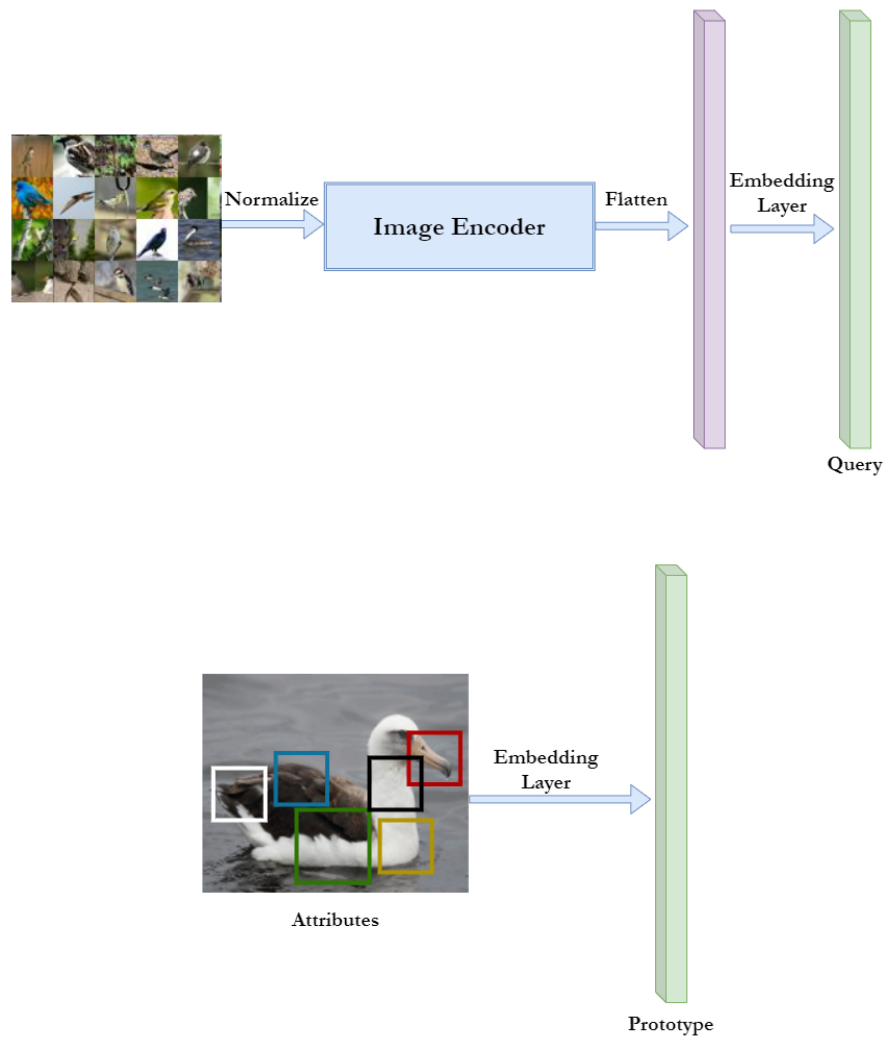


Figure 3: Zero-Shot Embedding Architecture

The code for this is not available online, so we create the entire process from scratch. We downloaded the data and the splits used in the paper directly from Wah et al. (2011) and created a custom dataloader so that it can be incorporated with the prototypical batch function, inspired by Sha-Lab (2018). We create the network architecture from scratch defining the image and attribute encoders. Indeed, there is significant computational overhead when passing cropped and transformed copies of images through GoogleNet. Since it is frozen and the transformations remain the same for every epoch, we save significant compute by caching transformed and embedded images before training. We also modify the Prototypical Loss function so that it can directly use embedded attributes to create prototypes rather than processing support examples as in the few-shot case.

We follow the training methodology described in the paper, with each training episode comprising 50 classes and 10 query images per class. The paper does not specify whether they freeze the image encoder, however, for the purposes of our experiments we do. For optimizing the embedding layer, a learning rate of 10^{-4} and a weight decay of 10^{-5} were used for SGD with Adam. In the paper, early stopping guided by the validation loss was employed to determine the optimal number of epochs before retraining on both the training and validation datasets. In our implementation, we employed early stopping, measuring validation loss every 100 episodes with a patience of 10 measurements. The best validation loss was achieved at epoch 8, which was then used to retrain the entire model. To evaluate the model's performance, we randomly sample 1000 episodes from the test set and calculate the average accuracy across these episodes.

However, Table 4 shows we were unable to replicate the results reported in the paper, with our results falling short of their performance. There are a few possible reasons for this discrepancy. One possibility is that the authors may have fine-tuned the entire GoogleNet encoder by backpropagating through all its layers, whereas we were limited by our GPU memory and could not do so. Another potential reason could be that they used different checkpoints or weights for the GoogleNet model than we did.

Image Features	50-way Acc.
	0-shot
GoogleNet	43.7%

Table 4: CUB Results

As we were not able to replicate the results, we also experimented with a different backbone network as an extension. GoogleNet was introduced in 2014, and since then, there have been many advancements in visual neural networks. In particular, attention-based networks, such as the Vision Transformer (ViT) introduced in 2020 by Dosovitskiy et al. (2020), have shown remarkable performance on image tasks. As Prototypical Networks are essentially linear classifiers with the majority of predictive power coming from the backbone, we hypothesize that the more powerful ViT would perform better. For ViT, we normalize images across channels with means of [0.5, 0.5, 0.5] and standard deviations of [0.5, 0.5, 0.5]. Table 5 confirms that results far surpass those reported in the original paper, which highlights how more powerful pre-trained models can improve performance in the zero-shot regime.

Image Features	50-way Acc.
	0-shot
ViT	65.3%

Table 5: ViT CUB Results

6 Mahalanobis Distance

As an extension, another Bregman divergence that can be used is the Mahalanobis Distance, which, like the Euclidean distance, also corresponds to prototypes of Gaussian Clusters but the assumption of an identity covariance matrix is relaxed. Consequently, it is parameterized not only by the mean vector c_k but also by a covariance matrix Σ_k .

$$d(f_\phi(x), c_k, \Sigma_k) = (f_\phi(x) - c_k)^T \Sigma_k^{-1} (f_\phi(x) - c_k) \quad (9)$$

When we use the Mahalanobis Distance, we are no longer restricted to a linear decision boundary as we are with Euclidean distance when clusters have identical covariance matrices. When this assumption is relaxed, it can be shown that the decision boundary becomes quadratic with respect to embeddings $f_\phi(x)$, reinforcing how allowing flexibility for variable prototype covariances enhances the flexibility of the model.

$$\begin{aligned} d(f_\phi(x), c_k, \Sigma_k) - d(f_\phi(x), c_{k'}, \Sigma_{k'}) &= 0 \\ (f_\phi(x) - c_k)^T \Sigma_k^{-1} (f_\phi(x) - c_k) - (f_\phi(x) - c_{k'})^T \Sigma_{k'}^{-1} (f_\phi(x) - c_{k'}) &= 0 \\ f_\phi(x)^T (\Sigma_k^{-1} - \Sigma_{k'}^{-1}) f_\phi(x) + 2f_\phi(x)^T (\Sigma_{k'}^{-1} c_{k'} - \Sigma_k^{-1} c_k) + (c_k^T \Sigma_k^{-1} c_k - c_{k'}^T \Sigma_{k'}^{-1} c_{k'}) &= 0 \end{aligned} \quad (10)$$

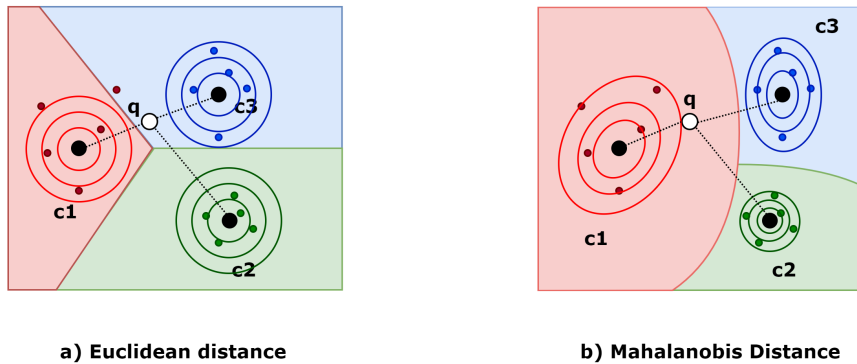


Figure 4: Illustration of support vectors and a test query in a two-dimensional embedding space. The Euclidean distance (left) assumes that each class-specific cluster has a unit-normal variance with linear decision boundary while the Mahalanobis distance (right) incorporates class-specific covariance matrices with a quadratic decision boundary (Figure adapted from Bateni, Goyal, et al. (2020).)

When defining the covariance matrix for each class, we face a challenge since we are working with unseen classes, which means we cannot define the covariance matrix ahead of time. Therefore, we need a method to estimate the covariance for each class during inference. Following the approach of Fort (2017), we modify the backbone network to predict not only an embedding for each sample but also a precision value for the embedding. This precision value allows the model to reflect the predictive power and quality of individual data points. Indeed, the precision of the sum of samples is equal to the sum of the individual sample precisions. This property allows us to construct the precision for the prototype using the individually predicted precisions.

$$\Sigma_k^{-1} = \sum_{i \in k} \Sigma_i^{-1} \quad (11)$$

Furthermore, this method enables us to create a better estimate of the prototype compared to simply using the MLE empirical mean. By utilizing the precisions as weights, we can construct a weighted average that has the least variance among all weighted averages, resulting in an improved prototype estimate.

$$c_k = \Sigma_k \sum_{i \in k} \Sigma_i^{-1} f_\phi(x_i) \quad (12)$$

We experimented with two methods to predict the precisions, and in both scenarios, we assume a diagonal covariance matrix, as Fort (2017) shows that predicting a dense covariance is unnecessarily complex. The first method involves predicting a single scalar precision value that represents every element along the diagonal, implying that the variance of all dimensions within the embedding is identical. To achieve this, we add an extra filter to the final convolutional block, which can be flattened and used as a scalar to represent the precision. The embedding size for Omniglot remains unchanged since it has a 1x1 output. However, for miniImageNet, which has a 5x5 output, the new embedding size becomes 1,624 dimensions when flattening. The second method predicts the entire diagonal precision, implying a different variance for each dimension of the embedding. This requires modifying the final convolutional block by doubling the number of filters, which, when flattened, doubles the vector output. The new outputs are then used as the precision values. When using this approach, the embeddings for both Omniglot and miniImageNet remain 64 and 1,600 dimensions, respectively.

For all predicted precisions, we apply the one-plus-softplus function, Equation 13, to ensure that the precision is greater than one, meaning that the model can only make data points less important.

$$S = 1 + \log(1 + e^{S_{\text{raw}}}) \quad (13)$$

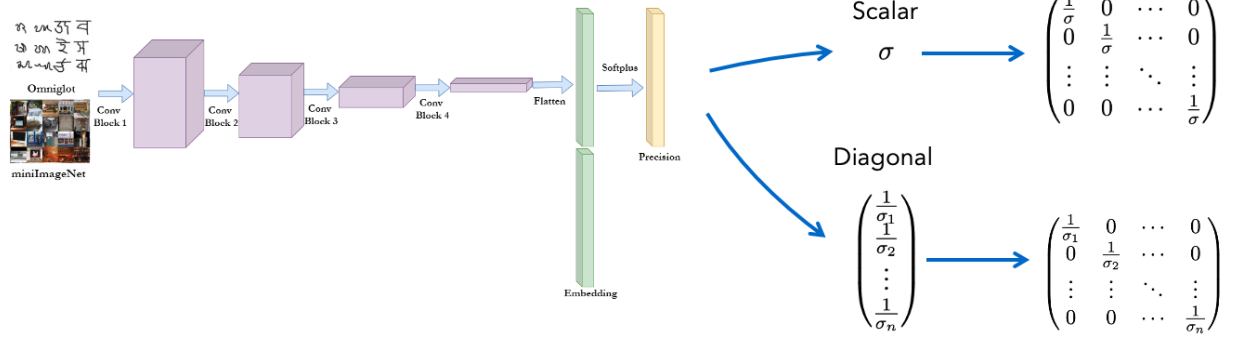


Figure 5: Precision Embedding Architecture

The original paper by Fort (2017) provides a TensorFlow implementation that inspired our network architecture and implementation. However, significant modifications were necessary to convert it to PyTorch, particularly when calculating the prototypical loss which requires collating the precisions for each prototype. We also created a function to compute the Mahalanobis Distance given prototypes, queries, and precisions. By utilizing element-wise multiplication operations, we can make the computation more efficient since the covariance matrices are diagonal, significantly improving the training speed. This computational efficiency is presumably the reason why the authors mentioned that using the full precision matrix was not necessary.

We experimented with this approach in the few-shot setting on both the Omniglot and Mini ImageNet tasks, using various combinations of shots and ways for training and testing. We employed the same optimization parameters as in the Euclidean Distance setting with the necessary modifications to the architecture.

Our results for Omniglot are within the same range as the Euclidean distance, but do not exceed it, which aligns with the findings by Fort (2017). However, they also experimented with a larger network and found improvements in that setting. The authors point out that Omniglot is not an ideal dataset for comparison since it already achieves near-perfect performance, and a more complex dataset is needed for further development of few-shot learning algorithms. They suggest that Gaussian prototypical networks might perform better on less homogeneous and noisier datasets, which are common in real-world applications. Consequently, we turned our attention to miniImageNet, a more complex dataset that the researchers did not attempt. We observed promising signs of improved performance when models were trained to classify with just one shot, although performance remained limited when trained on 5-shot tasks. Predicting a full diagonal precision matrix worked better than predicting a single scalar value, suggesting that this method is better with additional complexity, and perhaps our implementation or training process could be further optimized.

Distance Metric	Train Episodes			5-way Acc.		20-way Acc.	
	Shot	Query	Way	1-shot	5-shot	1-shot	5-shot
Euclidean	1	15	20	98.4%	99.6%	94.8%	98.6%
Scalar Mahalanobis	1	15	20	97.5%	99.4%	92.1%	97.9%
Diagonal Mahalanobis	1	15	20	98.2%	99.6%	94.8%	98.7%
Euclidean	5	15	20	98.3%	99.6%	94.4%	98.6%
Scalar Mahalanobis	5	15	20	97.8%	99.5%	93.2%	98.4%
Diagonal Mahalanobis	5	15	20	97.9%	99.6%	93.8%	98.6%

Table 6: Omniglot Results

Distance Metric	Train Episodes			5-way Acc.	
	Shot	Query	Way	1-shot	5-shot
Euclidean	1	15	30	48.3%	64.3%
Scalar Mahalanobis	1	15	30	48.8%	64.6%
Diagonal Mahalanobis	1	15	30	49.0%	64.8%
Euclidean	5	15	20	45.4%	67.7%
Scalar Mahalanobis	5	15	20	45.0%	66.8%
Diagonal Mahalanobis	5	15	20	45.2%	67.7%

Table 7: ImageNet Results

7 Prototype Refinement

Indeed, with the Euclidean or Mahalanobis distance, we can think of prototypes as centers of Gaussian clusters. Naturally, the accuracy of these clusters in representing different classes are limited as they are formed with limited numbers of supports in the few-shot setting. To address this, we can think about utilizing the queries themselves to refine our cluster representations in a form of unsupervised learning.

Inspired by Bateni, Barber, et al. (2022), we propose a soft-K assignment approach to update clusters. This method blends soft Bregman clustering for the E-step (assigning queries to prototypes) with standard Gaussian Mixture Model Expectation-Maximization updates for the M-step (updating prototypes based on new information). In the E-step, we assign responsibilities, $w_{j,k}$ to each class, k , for each sample. We assign a probability of 1 for supports, S_α as they are known and use the posterior probabilities for queries, Q_α . With Bregman divergences, the posterior probability is equal to the output classification probability, Equation 2.

$$w_{j,k} = \begin{cases} p(y_j = k | z_j) & z_j \in Q_\alpha \\ 1[y_j = k] & (z_j, y_j) \in S_\alpha \end{cases} \quad (14)$$

With the M-step, we use the standard updates for Gaussian mixtures. The updated mean of each prototype is the average of samples weighted by E-step responsibilities.

$$\mu_k = \frac{1}{n'_k} \sum_j w_{j,k} z'_j \quad (15)$$

Calculating the variance, however, poses a challenge. While we start with the standard responsibility-weighted outer product of deviations from the mean, direct use is infeasible in scenarios with limited samples per episode. When the number of samples is much smaller than the feature space dimensionality, the variance estimate becomes unreliable.

$$\Sigma_k = \frac{1}{n'_k} \sum_j w_{j,k} (z'_j - \mu_k)(z'_j - \mu_k)^T \quad (16)$$

To address this, we introduce regularization. We calculate a task-level covariance matrix, Σ' , based on all samples, then form a convex combination with the class-conditional covariance Σ_k . We select the weighting, λ_k to decrease as the number of support examples for a class grows, ensuring regularization is strongest when data is scarce. However, our experiments showed that the resulting matrix was still singular and not invertible for Mahalanobis distance calculations. Following suggestions by the author, we apply a diagonal perturbation using a parameter β , which allowed us to calculate the inverse. Their proposed value of 1 works well in practice.

$$\begin{aligned} Q_k &= \lambda \Sigma_k + (1 - \lambda) \Sigma' + \beta I, \\ \text{where } \Sigma' &= \frac{1}{\sum_k n'_k} \sum_j w_{j,k} (z'_j - \mu')(z'_j - \mu')^T, \\ \text{and } \lambda_k &= \frac{n_k}{n_k + 1} \end{aligned} \quad (17)$$

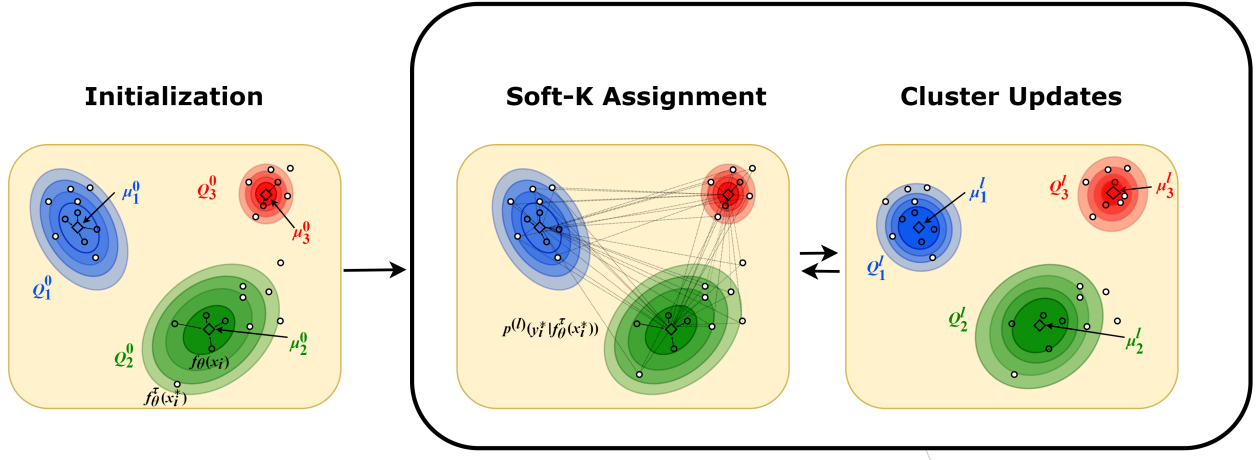


Figure 6: Illustration of the iterative clustering process using a soft k-means algorithm with Mahalanobis distance. During the *Initialization* phase (left), initial cluster centroids μ_k^0 , and optionally covariances, Σ_k^0 , are established for each cluster k . Subsequently, during the *Soft-K Assignment* (middle), data points are softly assigned to clusters based on the Mahalanobis distance, influencing cluster membership probabilities $p^{(l)}(y_i^* | f_\theta^T(x_i^*))$. Finally, for determining the *Cluster Updates*, the cluster centroids μ_k^l and covariances, Σ_k^l , are recalculated, reflecting the soft assignments (right). These steps are repeated iteratively to refine the cluster centroids and the assignment of data points until convergence. (Illustration adapted from Bateni, Barber, et al. (2022).)

We are inspired by the source code for Transductive CNAPS by Bateni, Barber, et al. (2022) who motivate this method, however, we found that it could not be adapted to our implementation of Prototypical Networks, and a rewrite was necessary. Particularly, significant modifications to the Prototypical Loss function are necessary to enable clustering updates after calculating prototypes. We also needed to introduce a new function that calculates the Mahalanobis Distance using a full covariance matrix, without the simplification of a diagonal precision that allows for direct element-wise multiplication. This process becomes computationally more demanding, as it also requires inverting the Covariance Matrix estimated from the E-Step for use in the M-Step.

Indeed, implementation of this method is highly flexible and there are numerous ways to initialize clusters. The first two methods leverage pre-trained networks and only apply clustering during testing. Firstly, we can utilize the trained Euclidean Distance network. The first update requires estimating the covariance matrix without any prior information. In the first E-Step, we assign no probabilities to the query and instead base all updates on the support samples. Consequently, the first M-Step involves calculating the initial covariance matrix using only the support samples. Secondly, we can directly use the predicted precisions from the trained Mahalanobis Distance network, using the predicted diagonal precision as the first estimate of the covariance matrix. For all experiments, we specify a maximum of 5 updates or stop the updates when the class assignments remain unchanged.

We observe that leveraging query data to improve prototype estimates enhances performance across all distance metrics and training parameters, demonstrating that this approach works as an out-of-the-box solution to improve performance at test time without significant investment. Previously, the Euclidean distance metric yielded the best performance on Omniglot. However, with the introduction of the clustering methodol-

ogy, the Mahalanobis network now outperforms the Euclidean distance for both 1-Shot and 5-Shot learning tasks. This indicates that the precision predicted by the Mahalanobis Network was valuable for Omniglot but required refinement. It also demonstrates that starting with a predicted precision is more advantageous than starting without one. On the other hand, for miniImageNet, the Mahalanobis distance was already better than the Euclidean distance. Applying clustering on top does not provide significant improvements and only brings the Euclidean performance closer to the Mahalanobis network. This suggests that the Mahalanobis network's predicted precision for ImageNet was already of good quality and further refinement using clustering offers limited improvements.

Distance Metric	Train Episodes			Test Episodes	5-way Acc.		20-way Acc.	
	Shot	Query	Way		1-shot	5-shot	1-shot	5-shot
Euclidean	1	15	20	No	98.4%	99.6%	94.8%	98.6%
Euclidean	1	15	20	Yes	97.6%	99.2%	94.3%	98.5%
Scalar Mahalanobis	1	15	20	No	97.5%	99.4%	92.1%	97.9%
Scalar Mahalanobis	1	15	20	Yes	98.5%	99.4%	93.5%	98.2%
Diagonal Mahalanobis	1	15	20	No	98.2%	99.6%	94.8%	98.7%
Diagonal Mahalanobis	1	15	20	Yes	98.8%	99.6%	96.8%	98.7%
Euclidean	5	15	20	No	98.3%	99.6%	94.4%	98.6%
Euclidean	5	15	20	Yes	97.3%	99.1%	93.6%	98.4%
Scalar Mahalanobis	5	15	20	No	97.8%	99.5%	93.2%	98.4%
Scalar Mahalanobis	5	15	20	Yes	98.9%	99.5%	94.6%	98.5%
Diagonal Mahalanobis	5	15	20	No	97.9%	99.6%	93.8%	98.6%
Diagonal Mahalanobis	5	15	20	Yes	98.9%	99.7%	95.4%	98.8%

Table 8: Omniglot Results

Distance Metric	Train Episodes			Test Episodes	5-way Acc.	
	Shot	Query	Way		1-shot	5-shot
Euclidean	1	15	30	No	48.3%	64.3%
Euclidean	1	15	30	Yes	49.7%	61.8%
Scalar Mahalanobis	1	15	30	No	48.8%	64.6%
Scalar Mahalanobis	1	15	30	Yes	49.5%	65.0%
Diagonal Mahalanobis	1	15	30	No	49.0%	64.8%
Diagonal Mahalanobis	1	15	30	Yes	49.8%	64.3%
Euclidean	5	15	20	No	45.4%	67.7%
Euclidean	5	15	20	Yes	47.9%	66.1%
Scalar Mahalanobis	5	15	20	No	45.0%	66.8%
Scalar Mahalanobis	5	15	20	Yes	48.9%	67.4%
Diagonal Mahalanobis	5	15	20	No	45.2%	67.7%
Diagonal Mahalanobis	5	15	20	Yes	48.7%	67.5%

Table 9: ImageNet Results

The third method of initialization is inspired by (Bateni, Goyal, et al. 2020), involving analytically estimating the covariance matrix with Equation 17 during training and incorporating this operation during back-propagation. Indeed, they suggest that this is beneficial over restrictios to just a diagonal covariance. When incorporating clustering, Bateni, Barber, et al. (2022) recommends just using a single step during training as applying numerous iterative updates on features which are noisy in the start would cause instabilities during training. We experimented with just a single clustering step during training and discovered that results were poor for Omniglot, suggesting that directly predicting diagonal precisions are more robust. We were not able to experiment with this method on ImageNet as our implementation is not efficient enough, as back-propagating through an inversion is a very expensive operation, particularly when working with large embedding spaces and we do not have sufficient GPU memory.

Distance Metric	Train Episodes				5-way Acc.		20-way Acc.	
	Shot	Query	Way	Clustered	1-shot	5-shot	1-shot	5-shot
Euclidean	1	15	20	No	98.4%	99.6%	94.8%	98.6%
Euclidean	1	15	20	Yes	92.3%	97.8%	80.5%	93.7%
Euclidean	5	15	20	No	98.3%	99.6%	94.4%	98.6%
Euclidean	5	15	20	Yes	96.9%	99.2%	93.3%	98.1%

Table 10: Omniglot Results

8 Discussion

Computationally very expensive to backpropagate clustering on the ImageNet dataset and ran out of memory so we did not attempt it. Perhaps the clustering steps are not coded very efficiently with vectorization, pawrticularly the Mahalanobis distance which was easy to do when diagonal sigma but not when it is full sigma. Perhaps we would need to experiment with techniques like using multiple GPUs, ASK GPT

Also could experiment with zero shot with including Sigma. We would predict a sigma for every class attribute to support the prototype. However, we would need to come up with a way of estimating the sigma in the first step since we can not similarly estimate it based on the uncertainty in the support samples so would probably need to predict it from the network itself at the start

Furthermore, the dataset Omniglot is already near perfect and to judge the performance of models, we would probably want to look into bigger datasets like ImageNet which could be a better judge of performance. Furthermore, it seems like the power of the method is in the backbone, since ViT made it so much better, and we could experiment with that for few-shot too. We could also experiment with making it more complicated as right now, it is just learning a linear mapping to move the embeddings in the same domain, but perhaps we could learn more layers to adapt the pre-trained model to the task specifically. This task can also be applied to other non-image related task such as text classification or etc and can be experimented there.

We can also experiment with fine-tuning parameters of the clustering. In the paper, they say that having too many steps is actually bad, however, we did not look into how many steps are being performed. They quote: We do see a decline in performance with a higher number of steps; this suggests that while our refinement criteria can be effective at performing different number of steps depending on the task, it can potentially lead to over-fitting, justifying the need for a well chosen maximum number of steps.

References

- Bateni, Peyman, Jarred Barber, et al. (2022). “Enhancing few-shot image classification with unlabelled examples”. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 2796–2805.
- Bateni, Peyman, Raghav Goyal, et al. (2020). “Improved few-shot visual classification”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14493–14502.
- Ciregan, Dan, Ueli Meier, and Jürgen Schmidhuber (2012). “Multi-column deep neural networks for image classification”. In: *2012 IEEE conference on computer vision and pattern recognition*. IEEE, pp. 3642–3649.
- Ciriello, Daniele (2018). *Prototypical Networks for Few-shot Learning PyTorch Implementation*. GitHub repository. MIT License. URL: <https://github.com/orobix/Prototypical-Networks-for-Few-shot-Learning-PyTorch>.
- Dosovitskiy, Alexey et al. (2020). “An image is worth 16x16 words: Transformers for image recognition at scale”. In: *arXiv preprint arXiv:2010.11929*.
- Finn, Chelsea, Pieter Abbeel, and Sergey Levine (2017). “Model-agnostic meta-learning for fast adaptation of deep networks”. In: *International conference on machine learning*. PMLR, pp. 1126–1135.
- Fort, Stanislav (2017). “Gaussian prototypical networks for few-shot learning on omniglot”. In: *arXiv preprint arXiv:1708.02735*.
- Graves, Alex, Greg Wayne, and Ivo Danihelka (2014). “Neural turing machines”. In: *arXiv preprint arXiv:1410.5401*.
- Koch, Gregory, Richard Zemel, Ruslan Salakhutdinov, et al. (2015). “Siamese neural networks for one-shot image recognition”. In: *ICML deep learning workshop*. Vol. 2. 1. Lille.
- Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E Hinton (2012). “Imagenet classification with deep convolutional neural networks”. In: *Advances in neural information processing systems* 25.
- Lake, Brenden et al. (2011). “One shot learning of simple visual concepts”. In: *Proceedings of the annual meeting of the cognitive science society*. Vol. 33. 33.
- Lake, Brenden M, Ruslan Salakhutdinov, and Joshua B Tenenbaum (2015). “Human-level concept learning through probabilistic program induction”. In: *Science* 350.6266, pp. 1332–1338.
- Liu, Yanbin et al. (2018). “Learning to propagate labels: Transductive propagation network for few-shot learning”. In: *arXiv preprint arXiv:1805.10002*.
- Liu, Yaoyao (2019). *mini-imagenet-tools*. GitHub repository. MIT License. URL: <https://github.com/yaoyao-liu/mini-imagenet-tools>.
- Parnami, Archit and Minwoo Lee (2022). “Learning from few examples: A summary of approaches to few-shot learning”. In: *arXiv preprint arXiv:2203.04291*.
- Perez, Ethan et al. (2018). “Film: Visual reasoning with a general conditioning layer”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1.
- Ravi, Sachin and Hugo Larochelle (2016). “Optimization as a model for few-shot learning”. In: *International conference on learning representations*.
- Reed, Scott et al. (2016). “Learning deep representations of fine-grained visual descriptions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 49–58.
- Requeima, James et al. (2019). “Fast and flexible multi-task classification using conditional neural adaptive processes”. In: *Advances in Neural Information Processing Systems* 32.
- Santoro, Adam et al. (2016). “Meta-learning with memory-augmented neural networks”. In: *International conference on machine learning*. PMLR, pp. 1842–1850.

- Sha-Lab (2018). *FEAT*. GitHub repository. MIT License. URL: <https://github.com/Sha-Lab/FEAT>.
- Simonyan, Karen and Andrew Zisserman (2014). “Very deep convolutional networks for large-scale image recognition”. In: *arXiv preprint arXiv:1409.1556*.
- Snell, Jake, Kevin Swersky, and Richard Zemel (2017). “Prototypical networks for few-shot learning”. In: *Advances in neural information processing systems* 30.
- Sung, Flood et al. (2018). “Learning to compare: Relation network for few-shot learning”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1199–1208.
- Vinyals, Oriol et al. (2016). “Matching networks for one shot learning”. In: *Advances in neural information processing systems* 29.
- Wah, C. et al. (2011). *The Caltech-UCSD Birds-200-2011 Dataset*. Tech. rep. CNS-TR-2011-001. California Institute of Technology.
- Welinder, Peter et al. (2010). “Caltech-UCSD birds 200”. In.